

A The Illustrating Example: SSA and Constraints

The following table corresponds to Figure 1. The “Execution” column lists the execution path “entry $\rightarrow$ B1 $\rightarrow$ B3 $\rightarrow$ B1 $\rightarrow$ B2 $\rightarrow$ exit”. The “SSA Statement” column shows the sequential SSA representation of Figure 1b. The remaining columns are the constraints collected to ensure the generated function is well-defined and deterministically terminates according to the given execution path. In this table, all computations involved in constraints are mathematical and do not overflow or underflow — the only UB considered in this example besides uninitialized scalars.

Execution	SSA Statement	Computation	Definedness	Control Flow
entry	int a_0 = s1, c_0 = s2;	$b_0 = s_0$ $a_0 = s_1$ $c_0 = s_2$	$\text{INT_MIN} \leq s_0 \leq \text{INT_MAX}$ $\text{INT_MIN} \leq s_1 \leq \text{INT_MAX}$ $\text{INT_MIN} \leq s_2 \leq \text{INT_MAX}$	—
B1	c_1 = a_0 * s3 + b_0 * s4 + s5;	$t_1 = a_0 \times s_3$ $t_2 = b_0 \times s_4$ $t_3 = t_1 + t_2$ $c_1 = t_3 + s_5$	$\text{INT_MIN} \leq s_3 \leq \text{INT_MAX}$ $\text{INT_MIN} \leq s_4 \leq \text{INT_MAX}$ $\text{INT_MIN} \leq s_5 \leq \text{INT_MAX}$ $\text{INT_MIN} \leq t_1 \leq \text{INT_MAX}$ $\text{INT_MIN} \leq t_2 \leq \text{INT_MAX}$ $\text{INT_MIN} \leq t_3 \leq \text{INT_MAX}$ $\text{INT_MIN} \leq c_1 \leq \text{INT_MAX}$	—
	if (c_1 * s6 + s7 <= 0) goto B3;	$t_4 = c_1 \times s_6$ $t_5 = t_4 + s_7$	$\text{INT_MIN} \leq s_6 \leq \text{INT_MAX}$ $\text{INT_MIN} \leq s_7 \leq \text{INT_MAX}$ $\text{INT_MIN} \leq t_4 \leq \text{INT_MAX}$ $\text{INT_MIN} \leq t_5 \leq \text{INT_MAX}$	$t_5 \leq 0$
B3	a_1 = c_1 * s8 + s9;	$t_6 = c_1 \times s_8$ $a_1 = t_6 + s_9$	$\text{INT_MIN} \leq s_8 \leq \text{INT_MAX}$ $\text{INT_MIN} \leq s_9 \leq \text{INT_MAX}$ $\text{INT_MIN} \leq t_6 \leq \text{INT_MAX}$ $\text{INT_MIN} \leq a_1 \leq \text{INT_MAX}$	—
	goto B1;	—	—	—
B1	c_2 = a_1 * s3 + b_0 * s4 + s5;	$t_7 = a_1 \times s_3$ $t_8 = t_7 + t_2$ $c_2 = t_8 + s_5$	$\text{INT_MIN} \leq t_7 \leq \text{INT_MAX}$ $\text{INT_MIN} \leq t_8 \leq \text{INT_MAX}$ $\text{INT_MIN} \leq c_2 \leq \text{INT_MAX}$	—
	if (c_2 * s6 + s7 <= 0) goto B3;	$t_9 = c_2 \times s_6$ $t_{10} = t_9 + s_7$	$\text{INT_MIN} \leq t_9 \leq \text{INT_MAX}$ $\text{INT_MIN} \leq t_{10} \leq \text{INT_MAX}$	$t_{10} > 0$
B2	goto exit;	—	—	—
exit	return a_1 + b_0 + c_2;	$t_{11} = a_1 + b_0$ $s_{10} = t_{11} + c_2$	$\text{INT_MIN} \leq t_{11} \leq \text{INT_MAX}$ $\text{INT_MIN} \leq s_{10} \leq \text{INT_MAX}$	—

B Full Semantic Encoding Rules

The following table extends Table 1 to support the entire grammar of SYMLANG. In this table: `INT_MIN` and `INT_MAX` indicate the smallest and largest signed integer value, respectively; `i32_const(·)` creates a 32-bit integer free variable with a given name; `sext1(·)` extends the given value by one bit, e.g., `sext1(0b01) = 0b001`; `round0(·)` casts a float value into an integer by rounding it toward zero; `next( $\pi$ )` gives the next basic block to be visited along path π ; $\varphi[\![\cdot]\!]$ converts a symbolic entity to its SMT formula. Regarding the `RETURN` rule, the constraints are encoded based on the checksum function employed; the table uses a simple addition as an example, but in practice, CRC32 is used. Furthermore, although we classified rules into three categories, some computation and control-flow rules also imply definedness. For example: the computation rules of `DECLARE` indicate that all scalars are initialized before use; the computation rules of `*-EXPR` imply a well-defined, left-to-right evaluation order; the grammar itself prevents invalid scalars; the terminating control flow ensures that there are no infinite loops without side effects, etc.

Rule	Grammar	Computation	Definedness	Termination
SYMBOL	s_i	$\varphi[\![s_i]\!] = \text{i32_const}(\text{name}:s_i)$	$\varphi[\![s_i]\!] \in [\text{INT_MIN}, \text{INT_MAX}]$	–
LOC-VAR	v_i	$\varphi[\![v_i]\!] = \text{i32_const}(\text{name}:v_i)$	$\varphi[\![v_i]\!] \in [\text{INT_MIN}, \text{INT_MAX}]$	–
DECLARE	$v_i := s_j$	$\varphi[\![v_i]\!] = \varphi[\![s_j]\!]$	–	–
ADD-TERM	$v_i + s_j$	$\varphi[\![v_i]\!] + \varphi[\![s_j]\!]$	$\text{sext1}(\varphi[\![v_i]\!]) + \text{sext1}(\varphi[\![s_j]\!]) \in [\text{INT_MIN}, \text{INT_MAX}]$	–
SUB-TERM	$v_i - s_j$	$\varphi[\![v_i]\!] - \varphi[\![s_j]\!]$	$\text{sext1}(\varphi[\![v_i]\!]) - \text{sext1}(\varphi[\![s_j]\!]) \in [\text{INT_MIN}, \text{INT_MAX}]$	–
MUL-TERM	$v_i * s_j$	$\varphi[\![v_i]\!] \times \varphi[\![s_j]\!]$	$\text{sext1}(\varphi[\![v_i]\!]) \times \text{sext1}(\varphi[\![s_j]\!]) \in [\text{INT_MIN}, \text{INT_MAX}]$	–
DIV-TERM	v_i / s_j	$\text{round0}(\varphi[\![v_i]\!] \div \varphi[\![s_j]\!])$	$\varphi[\![s_j]\!] \neq 0$ $\varphi[\![v_i]\!] \neq \text{INT_MIN} \vee \varphi[\![s_j]\!] \neq -1$ $\varphi[\![v_i]\!] \div \varphi[\![s_j]\!] \times \varphi[\![s_j]\!] = \varphi[\![v_i]\!]$	–
REM-TERM	$v_i \% s_j$	$\varphi[\![v_i]\!] - \varphi[\![v_i/s_j]\!] \times \varphi[\![s_j]\!]$	$\varphi[\![s_j]\!] \neq 0$ $\varphi[\![v_i]\!] \neq \text{INT_MIN} \vee \varphi[\![s_j]\!] \neq -1$ $\varphi[\![v_i]\!] \div \varphi[\![s_j]\!] \times \varphi[\![s_j]\!] = \varphi[\![v_i]\!]$	–
ADD-EXPR	$\varepsilon_i + \tau_j$	$\varphi[\![\varepsilon_i]\!] + \varphi[\![\tau_j]\!]$	$\text{sext1}(\varphi[\![\varepsilon_i]\!]) + \text{sext1}(\varphi[\![\tau_j]\!]) \in [\text{INT_MIN}, \text{INT_MAX}]$	–
SUB-EXPR	$\varepsilon_i - \tau_j$	$\varphi[\![\varepsilon_i]\!] - \varphi[\![\tau_j]\!]$	$\text{sext1}(\varphi[\![\varepsilon_i]\!]) - \text{sext1}(\varphi[\![\tau_j]\!]) \in [\text{INT_MIN}, \text{INT_MAX}]$	–
ASSIGN	$v_i := \varepsilon_j$	$\varphi[\![v_i]\!] = \varphi[\![\varepsilon_j]\!]$	–	–
ABS-JMP	goto L	–	–	–
CON-JMP	if(ε) goto L_1 else goto L_2	–	–	$\varphi[\![\varepsilon]\!] \neq 0$ if $L_1 = \text{next}(\pi)$ $\varphi[\![\varepsilon]\!] = 0$ if $L_2 = \text{next}(\pi)$
RETURN	return $\sum_{i=0}^n v_i$	$\varphi[\![o]\!] = \sum_{i=0}^n \varphi[\![v_i]\!]$	$\forall j \in [1, n].$ $\sum_{i=0}^j \text{sext1}(\varphi[\![v_i]\!]) \in [\text{INT_MIN}, \text{INT_MAX}]$	–

C More Experimental Data

This section provides more data for our evaluation, such as the scalability (Figure 7) or the coverage experiment (Table 4). For the former, we include more independent parameters and more configurations for each independent parameter, as well as more suggested configurations. For the latter, we present the optimization-related GCC or LLVM files where our coverage is better.

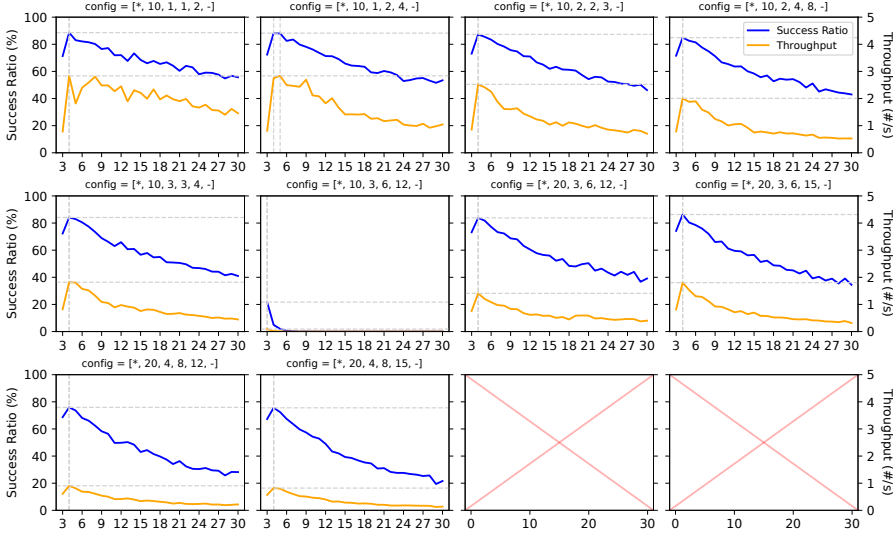


Fig. 9. Independent Parameter: #BBL of REIFY°. The configuration $[*, 10, 3, 6, 12, -]$ is invalid, as it violates the requirement that the number of variables used per condition (12) must not exceed the total number of variables (10), rendering the plot incorrect.

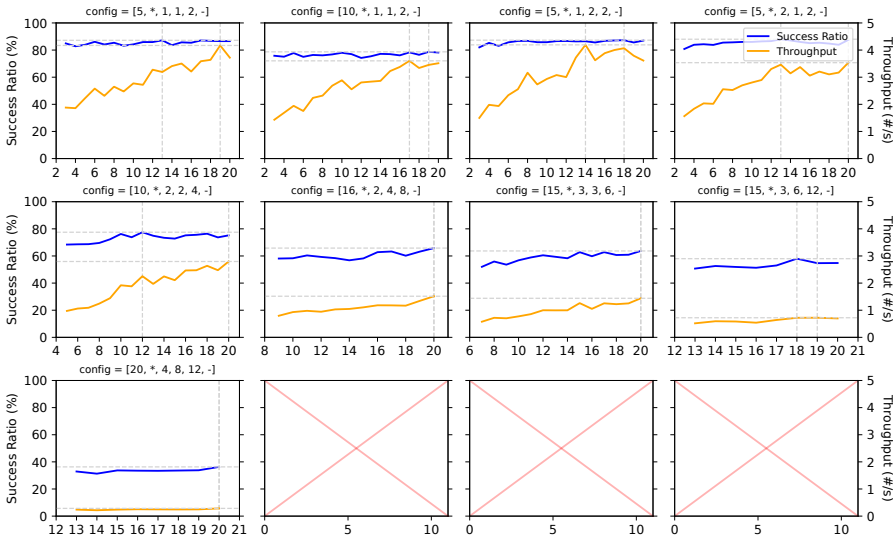


Fig. 10. Independent Parameter: #VAR of REIFY°

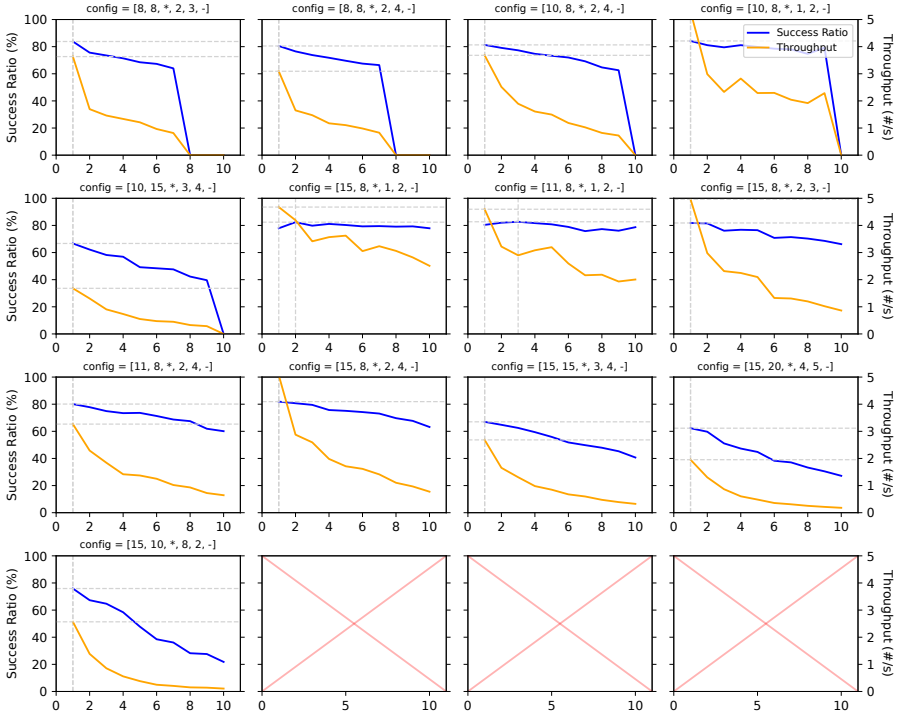


Fig. 11. Independent Parameter: #ASS of REIFY^o. This plot shows that #ASS is another major parameter influencing the scalability and performance of REIFY^o, alongside #BBL. This is expected, as an increase in assignments per basic block raises the number of constraints by at least the number of basic blocks. For any configuration, once the number of assignments exceeds a certain threshold, the success ratio drops sharply. This is because the additional assignments generate a dense set of constraints over a limited set of variables, increasing the likelihood of being unsatisfiable. According to it, choosing 2–5 assignments per basic block leads to a better balance among all evaluated configurations.

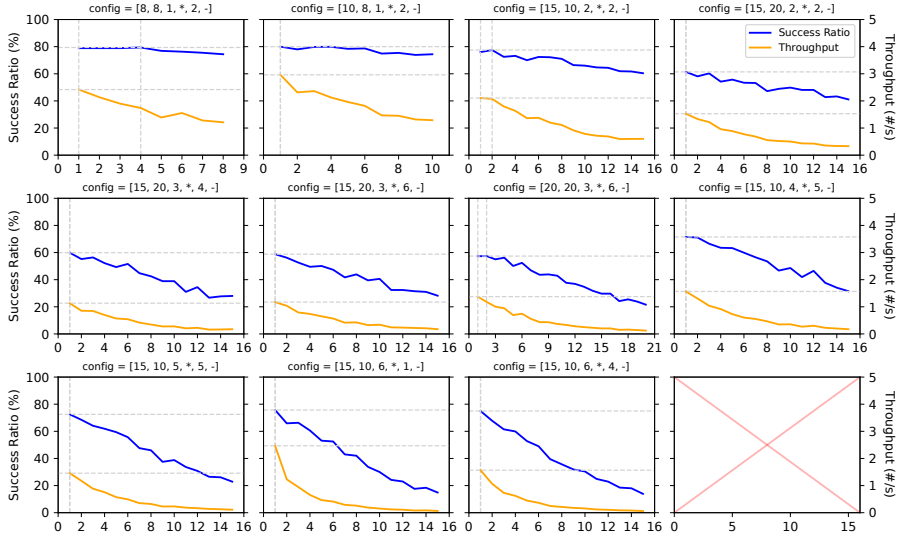


Fig. 12. Independent Parameter: #TERA of REIFY°. Increasing the number of variables used in each assignment strengthens the correlations between variables and constrains the solution space, leading to negative influence.

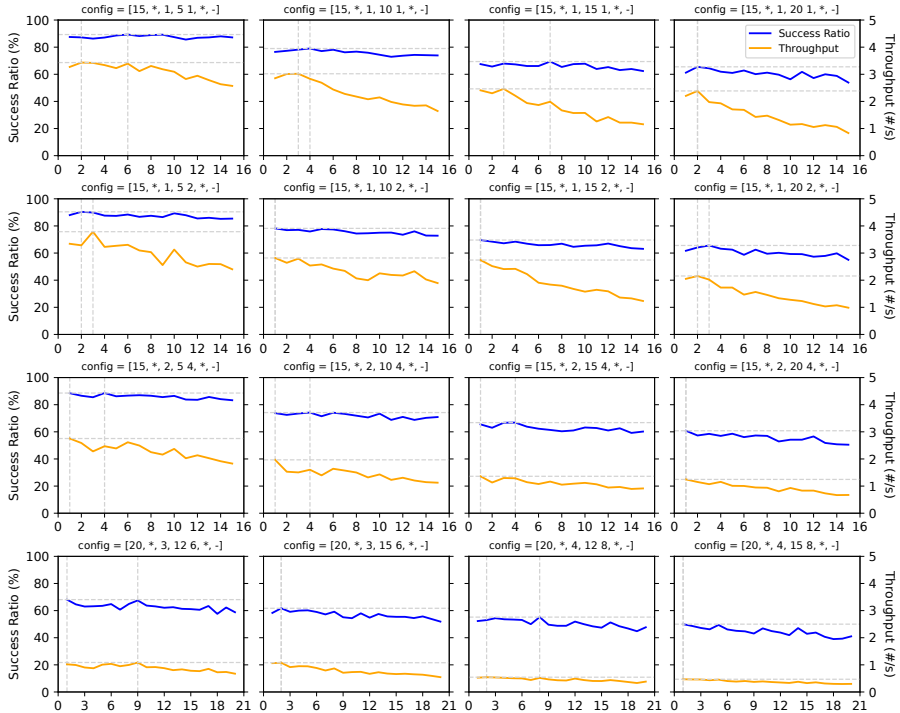


Fig. 13. Independent Parameter: #TERC of REIFY°. Its influence stems from the same mechanism as #TERA, but #TERC has a smaller effect, since the former impacts all assignments while the latter only affects control flow.

Table 5. Configurations and their success rate and throughput. The data is sorted via success ratio first, then throughput. For a configuration, “*” denotes the independent parameter; “-” indicates a non-applicable parameter; numeric values represent controlled parameters; “a” means any value.

(a) Configs #1–#25				(b) Configs #26–#50				(c) Configs #51–#70			
#	Configuration	SUCC	Thr.	#	Configuration	SUCC	Thr.	#	Configuration	SUCC	Thr.
1	[15,8,2,2,3,a]	81.5	2.98	26	[10,10,2,2,3,a]	74.7	1.64	51	[10,8,6,2,4,a]	72.0	1.19
2	[15,8,2,2,4,a]	80.7	2.88	27	[10,8,4,2,4,a]	74.7	1.61	52	[8,20,3,6,15,a]	72.0	1.12
3	[15,8,3,2,4,a]	79.5	2.59	28	[15,8,6,2,4,a]	74.2	1.62	53	[15,10,2,4,8,a]	71.9	1.50
4	[10,8,2,2,4,a]	79.1	2.52	29	[15,10,2,4,4,a]	74.2	1.60	54	[8,8,4,2,4,a]	71.8	1.17
5	[8,10,2,2,3,a]	78.3	1.61	30	[15,10,2,4,6,a]	74.1	1.64	55	[15,10,2,4,5,a]	71.5	1.40
6	[11,8,2,2,4,a]	77.8	2.29	31	[10,11,2,2,4,a]	73.8	1.88	56	[8,8,4,2,3,a]	71.4	1.34
7	[10,12,2,2,4,a]	77.5	2.25	32	[8,8,3,2,4,a]	73.8	1.47	57	[15,8,7,2,3,a]	71.4	1.31
8	[15,10,2,2,2,a]	77.5	2.08	33	[10,19,2,2,4,a]	73.7	2.47	58	[11,8,6,2,4,a]	71.3	1.25
9	[10,8,3,2,4,a]	77.3	1.90	34	[11,8,5,2,4,a]	73.6	1.37	59	[11,10,2,2,3,a]	71.2	1.44
10	[15,8,4,2,3,a]	76.8	2.25	35	[15,10,2,4,3,a]	73.5	1.50	60	[9,10,2,4,8,a]	71.2	1.24
11	[15,8,5,2,3,a]	76.5	2.10	36	[8,8,3,2,3,a]	73.5	1.46	61	[15,10,4,2,5,a]	71.1	1.31
12	[8,8,2,2,4,a]	76.5	1.65	37	[10,14,2,2,4,a]	73.4	2.25	62	[12,10,2,2,3,a]	71.0	1.34
13	[10,18,2,2,4,a]	76.4	2.64	38	[15,10,2,4,10,a]	73.4	1.43	63	[15,10,2,4,12,a]	71.0	1.31
14	[10,10,2,2,4,a]	76.2	1.92	39	[11,8,4,2,4,a]	73.4	1.42	64	[15,10,2,8,2,a]	71.0	1.11
15	[15,8,3,2,3,a]	76.1	2.32	40	[8,10,3,3,4,a]	73.4	1.32	65	[15,10,2,4,15,a]	70.9	1.12
16	[15,8,4,2,4,a]	75.7	1.98	41	[10,8,5,2,4,a]	73.3	1.50	66	[15,8,6,2,3,a]	70.8	1.33
17	[9,10,2,2,3,a]	75.7	1.60	42	[15,10,2,4,2,a]	73.2	1.63	67	[15,10,2,4,9,a]	70.6	1.32
18	[10,17,2,2,4,a]	75.6	2.47	43	[15,10,2,4,7,a]	73.2	1.57	68	[15,8,8,2,3,a]	70.3	1.20
19	[8,8,2,2,3,a]	75.6	1.70	44	[15,8,7,2,4,a]	73.1	1.41	69	[15,10,2,4,14,a]	70.3	1.14
20	[10,20,2,2,4,a]	75.2	2.80	45	[10,15,2,2,4,a]	72.8	2.11	70	[15,10,2,5,2,a]	70.0	1.36
21	[10,16,2,2,4,a]	75.2	2.47	46	[15,10,2,3,2,a]	72.5	1.80				
22	[15,8,5,2,4,a]	75.1	1.71	47	[15,10,2,4,2,a]	72.5	1.53				
23	[10,13,2,2,4,a]	74.9	1.97	48	[15,10,2,6,2,a]	72.4	1.38				
24	[11,8,3,2,4,a]	74.9	1.84	49	[10,9,2,2,4,a]	72.3	1.44				
25	[8,10,2,4,8,a]	74.8	1.48	50	[15,10,2,7,2,a]	72.2	1.20				

Table 6. The top 10 optimization-related files where REIFY covers more lines of code (left) or exclusively covers more functions (right). In the left table, the column “%” indicates REIFY’s improvement in line coverage compared with either Csmith or YARPGen, and in the right table, the column “#” denotes the number of exclusively covered functions by REIFY compared with either Csmith or YARPGen.

(a) Better Coverage				(b) Exclusive Coverage			
GCC	%	LLVM	%	GCC	#	LLVM	#
ipa-cp.cc	37.6	DivRemPairs.cpp	39.5	ipa-cp.cc	85	SROA.cpp	44
tree-sra.cc	36.1	CFG.cpp	1.4	tree-sra.cc	57	FunctionSpecializ.cpp	22
ipa-param-manip.cc	37.2	MemoryDepAnalysis.cpp	4.2	ipa-prop.cc	39	DeadStoreElimination.cpp	15
ipa-sra.cc	20.7	DeadArgElimination.cpp	4.5	ipa-param-manip.cc	37	InlineCost.cpp	15
ipa-fnsummary.cc	11.8	AliasAnalysis.cpp	5.2	cfgexpand.cc	30	CFG.cpp	14
tree-inline.cc	9	MemCpyOptimizer.cpp	1.7	ipa-sra.cc	21	ArgumentPromotion.cpp	13
ipa-split.cc	29.3	EarlyCSE.cpp	1.8	tree-eh.cc	16	SCCPSolver.cpp	11
ipa-modref.cc	8.1	LoopInfo.cpp	2.1	cgraphclones.cc	14	MemCpyOptimizer.cpp	11
ipa-cp.cc	6.3	BranchProbInfo.cpp	1.8	df-problems.cc	11	GlobalOpt.cpp	11
ipa-inline.cc	11.6	SROA.cpp	22.5	cgraph.cc	11	BranchProbInfo.cpp	10

Received 2025-11-14; accepted 2026-04-03